BAC Interface Design Guide

Build a CubeSat – Interface & interaction standard Version 1.0, 2026-10-27 · Authors: Mänu (design owner) and Luiz Guilherme Ito (Gui)

This guide defines how BAC interfaces communicate and behave across technical and non-technical surfaces: websites, the BAC Ground console, product/configuration interfaces, dashboards, notebooks, terminal applications, and physical indicators.

> **Authority.** The guides live at https://github.com/buildacubesat/bac-design. The **BAC Identity Guide** is authoritative for brand identity, voice and tone, canonical colours/tokens, typography, form, imagery, and the Ops/Mission communication modes. This **Interface Design Guide** is authoritative for interaction, semantic state usage, accessibility, motion, components, data visualisation, feedback/progress, interface layout patterns, terminal presentation, and physical indicator semantics. The **BAC Project & Tooling Guide** is authoritative for implementation and engineering conventions. Interface rules should reference Identity tokens rather than redefine them; tooling implementations should reference this guide rather than invent local UX patterns.
>
> **Reference implementation.** Since 1.0 the interface language follows the BAC Ground mission-operations console, whose state system, controls and density were adopted here.
>
> **Brand marks.** Interfaces show the BAC logo as the Identity Guide §8 defines it; its use by others – in their own products, apps or forks – follows the **BAC Trademark Policy**.

0. What this is, in one paragraph

BAC interfaces are **clear, calm, technical, and legible**. They expose the information and actions needed for the current task, surface failures visibly, suppress routine noise, and remain understandable without relying on colour alone. Every state is shown three ways – colour, shape and word. The same principles apply whether the interface is a public storefront, the ground console, an internal pricing notebook, a terminal utility, or a row of LEDs on a PCB. Platform conventions may shape the implementation, but the semantic language stays consistent.

1. Core interface principles

- **Minimal surface, maximum clarity.** Show only what the user needs at each step. Suppress noise by default; surface detail on request or on failure.
- **Fail loudly, succeed quietly.** Errors must be visible and actionable. Successful routine operations need no ceremony.
- **Always show liveness.** Any operation that takes more than ~1 second without otherwise producing visible feedback must show progress or activity for its duration.
- **Progressive disclosure.** Keep the primary path simple; place verbose diagnostics, raw previews, and advanced controls behind secondary affordances.

- **Consistent semantics across media.** green = nominal/pass, yellow = caution/warning, red = alarm/fault/fail, blue = information/activity. The same meaning survives across web UI, the console, terminal output, reports, and physical indicators.
- **Colour carries state, so nothing else uses the status colours.** Decoration, branding and selection never borrow green, red or the info blue.
- **Platform-native when it improves comprehension.** BAC styling should not fight a host application. Blender's title-case UI is the documented example. Platform conventions may override presentation details, never semantic meaning or accessibility.
- **Open-source and inspectable.** Prefer clear labels, explicit state, human-readable values, and interfaces whose operation can be understood without hidden magic.
- **No decorative emoji.** Use only semantic glyphs defined by the Identity Guide.

2. Semantic state & accent application

All canonical colours and token values live in the **BAC Identity Guide** (executable copy: `tokens.json` in bac-utils). This guide defines what those tokens mean in interfaces.

2.1 The five states

State	Fill	Text ink (light / dark)	Shape	Word examples
Nominal	--bac-green #3FB950	#1A7F37 / #3FB950	Filled circle	NOMINAL, PASS, OK
Caution	--bac-yellow #F7D400	#7A5F00 / #F7D400	Triangle	CAUTION, WARN, DEGRADED
Alarm	--bac-red #E94538	#C9302C / #FF5A4F	Square	ALARM, FAULT, FAIL
Information	--bac-blue #004C8B	#0B5CAD / #4C9AFF	Diamond	INFO, COMMAND, BEACON
No data / planned	--chart-6 (#8A8A8A / #6B6B6B)	--text-muted	Hollow circle, dashed outline	NO DATA, PLANNED, NOT FITTED

- **A state is always colour, shape and word.** Nothing depends on colour alone. The glyphs ✓ × ! → (Identity Guide §2) remain the text equivalents in terminals and plain text.
- **Fill and ink.** Fills draw dots, shapes, bars and badges; text uses the ink of the current theme, which meets 4.5:1 on the page, on cards and on the matching tint. In light mode yellow is always a fill with dark text (7.2:1), never yellow text on white.
- **F Prime event severities** map onto the same shapes: WARNING_HI and FATAL square, WARNING_LO triangle, COMMAND diamond, ACTIVITY filled or hollow circle, DIAGNOSTIC dashed.

- **Status shapes are not icons.** They are a fixed glyph set (CSS shapes or inline SVG) owned by this section; icons come from Lucide (§6).

2.2 Accent and action colours

Role	Light	Dark	Interface meaning
Primary action	--btn-primary-bg: BAC Yellow, dark text	--btn-primary-bg: grey #353433, light text	The one dominant action per state
State accent	--accent-state: BAC Yellow	--accent-state: light grey #E5E5E5	Switch on, selected segment or tab, current stat card, progress fill
Caution	BAC Yellow	BAC Yellow	Warning, degraded, attention
Links, focus	#0B5CAD	#4C9AFF	Links and the focus ring

One standard dark mode. Dark mode has **no accent colour**: primary buttons are grey and state marks are neutral, so yellow in dark mode means caution and nothing else. The small yellow kicker bar before eyebrows is a brand mark and stays.

User accent setting. The state accent is one token, --accent-state. A product may offer the user a dark-mode accent setting (neutral, blue or yellow), provided state never depends on the accent colour alone: the switch knob's position, the selected segment's raised fill or outline weight, and the label carry it in every setting. The standard and the default are neutral.

One dominant yellow use. "One dominant use per state" means one visually dominant yellow surface or action. Multiple small semantic uses may coexist when they communicate real state, such as caution badges beside a primary action. The rule prevents competing focal points; it does not suppress legitimate status indicators.

No functional amber, no decorative status colours. The nebula colours are Mission/marketing accents defined by the Identity Guide; they never replace a status colour in functional interfaces and never appear in Ops UI.

Colour never stands alone. Pair semantic colour with a label, shape, glyph, position, stroke pattern, or other redundant channel. Use ✓ for pass, × for fail, ! for warning, → for progression, and , or · as a separator.

3. Themes, surfaces & dark mode

The **Ops/Mission choice belongs to the BAC Identity Guide** because it is a communication decision. Interfaces then apply the chosen identity mode without changing semantic behaviour.

- **Ops interfaces** are restrained: flat neutral surfaces, depth from background steps and hairlines (no shadows), monochrome hierarchy, status colour only where it means something, no atmospheric decoration.
- **Mission interfaces** may use the Identity Guide's dark heroes, nebula accents, recessed viewports, and expressive presentation surfaces, while controls and states still follow this guide.

- **Light and dark are both first-class.** GUI and web interfaces follow prefers-color-scheme by default and default to light when the system reports no preference – browsers do not distinguish "no preference" from light, so a dark fallback cannot be done in CSS. A [data-theme="light|dark"] override may be provided, and a theme toggle (the half-filled circle) where the product has a header bar.
- **Dark mode is one standard mode** (§2.2): grey primary buttons, neutral state marks, yellow only as caution. Input wells are a step *lighter* than the page in dark mode and a step darker in light mode.
- **Test both modes.** Do not design one mode and patch the other.
- **Terminal themes are user-owned.** Terminal tools do not implement light/dark mode; use the terminal profile in §10.
- **Fixed UI.** Sticky navigation, a floating header bar and pinned announcement bars are acceptable where they preserve context without obscuring content. Panel actions such as Explore/Export belong at the top-right of the panel they affect.
- **Width.** Reading pages use --container (1200px); dashboards and dense data views such as BAC Ground use --container-wide (1540px).

4. Motion & interaction

4.1 Motion – restrained and functional

```
--ease-standard: cubic-bezier(0.2, 0, 0.1, 1); /* most transitions */
--ease-out: cubic-bezier(0.16, 1, 0.3, 1);     /* entrances */
--dur-fast: 120ms; --dur-normal: 200ms; --dur-slow: 320ms;
```

Fades and small colour changes only – **no bounces, no lifts, no infinite loops on content.** A loading shimmer is the one permitted repeating motion, and only while data is loading. Always respect prefers-reduced-motion.

4.2 Interaction states

- **Hover.** Light primary: BAC Yellow → --btn-primary-hover (#EED228). Dark primary and all grey or ghost controls: one surface step up (--surface-hover, then --border-strong). Table rows: --surface-hover. Links underline. Cards do not lift and gain no shadow.
- **Press.** translateY(1px) – a small physical depress.
- **Selected / on.** --accent-state with --accent-state-text, plus the shape or position change that carries the state without colour.
- **Focus.** A 3 px solid --focus-ring outline with a 2 px offset (outline: 3px solid var(--focus-ring); outline-offset: 2px) – #0B5CAD in light mode (6.3:1 on --bg), #4C9AFF in dark mode (5.8:1 on --bg, 5.2:1 on cards), well above the 3:1 a focus indicator needs (WCAG 2.2, 1.4.11). No halo shadow.
- **Disabled.** --text-faint on the control's normal surface, no hover change, cursor not-allowed. Disabled text is exempt from contrast rules and must never be the only place information lives.

5. Accessibility

Accessibility is a requirement, not a polish step. Every BAC surface – Ops and Mission alike – meets these:

- **Contrast.** All interactive text/background pairs target **WCAG AA** (4.5:1 for body text, 3:1 for large text and UI glyphs). The Identity Guide records the measured ratios of the semantic tokens; --text-faint (3.3:1 in light mode) is for disabled and decorative text only.
- **Never colour alone.** Colour must never be the only carrier of status. Every state has its shape and word (§2.1) – a red field also says why; a pass chip also shows its filled circle and word; separate board LEDs are identified by silkscreen. Shared multi-colour LEDs additionally require a dynamic non-colour channel as defined in §12.3.
- **Focus.** Focus states must be visible on both light and dark surfaces. The 3 px solid --focus-ring outline of §4.2 is the default; never remove a focus outline without an equivalent replacement.
- **Charts.** Series must be distinguishable by **stroke style or marker shape** in addition to colour (see §8).
- **Tabular figures.** Numbers that are compared – tables, stat cards, telemetry, countdowns – are set in IBM Plex Mono with tabular figures so digits line up.
- **Motion.** Respect prefers-reduced-motion (see §4.1).

6. Components & control patterns

The component model below is the canonical BAC interface vocabulary. The visual tokens come from the BAC Identity Guide. Implementations may use different frameworks as long as their semantics and states remain equivalent; BAC Ground is the reference implementation.

Buttons

- Button – a pill (--radius-pill) with an uppercase IBM Plex Mono label (500, --ls-label 0.03em); variant: primary (--btn-primary-*: yellow in light, grey in dark; **one dominant action per state**) / secondary (outlined, transparent fill) / ghost / danger (alarm ink on the alarm tint); size: sm/md; leadingIcon / trailingIcon as inline Lucide SVG.
- IconButton – round icon-only sibling (theme toggle, menu, toolbar actions) with an **18px** Lucide icon, transparent until hover, then --surface-hover.

Forms (shared: uppercase mono label + 6px inputs + the focus ring of §4.2)

- Input – label, unit suffix (CHF/%/mm), hint, error (alarm border and ink, plus the message). Idle fill --surface-sunken.
- Select – native dropdown styled to match Input; options.
- Switch – on-state is --accent-state with the knob moved to the right (yellow in light, light grey in dark).

- SegmentedControl – a pill group for 2–4 short options; the selected segment takes --accent-state (light) or a raised grey segment with a stronger outline (dark). Use Select for longer lists.

Navigation

- Tabs – pills; the current tab is the dark filled pill (--text-strong on --text in light, inverted in dark), the others plain uppercase mono labels.

Feedback

- Chip (status pill) – colour + shape + word: the state's shape in its fill colour, the word in uppercase mono in the state's ink, a hairline border tinted with the state colour, optional tint fill. Variants follow §2.1: nominal, caution, alarm, info, no-data/planned (dashed, hollow).
- Tag – neutral pill; optional onRemove.
- Callout – block-level message on the matching tint with a tinted border; kind: note / info / success / caution / alarm. Reserve for messages the user must not skip ("fail loudly").

Data

- Card – --surface-card, 10px radius, hairline border, no shadow; eyebrow (· LABEL in uppercase mono), title, actions.
- StatCard – label (uppercase mono), value (mono, tabular), unit, sublabel; current = --accent-state outline; tone: default / nominal / alarm / info colours the value with the state's ink and pairs it with the state's shape.
- DataTable – mono uppercase header labels in --text-muted over a strong header rule, right-aligned numeric columns in mono tabular figures, per-column render for custom cells (chips, links).
- Hero card – see below.

Hero card

The one highlighted item on a view: the time-critical or primary-context fact the user should see first (BAC Ground's "Next pass"). Anatomy: an eyebrow, a large uppercase title in Nunito Sans, a short **red rule** under the title, mono details, an optional big mono figure (a countdown), chips or tabs, an optional progress bar.

- Fill --hero-bg (info blue #004C8B), text --hero-text (white, 8.7:1), rule --hero-rule (BAC Red).
- **At most one per view.** No other accent fills on it; chips on the card use white outlines and white text.
- **The red rule is decorative** – a brand detail carried over from the founder's video intro cards. It never signals an alarm, and an alarm on the card is shown with the alarm chip (shape and word) like everywhere else.
- The same card in light and dark mode.

Usage sketch

```
<Button variant="primary" leadingIcon={<CubeIcon/>}>Learn more</Button>
<Input label="Target margin" defaultValue="20" unit="%" />
<Chip state="nominal">Uploaded</Chip>
<StatCard label="USD export price" value="$1,517" sublabel="1'137.74 CHF excl. VAT" current />
<Callout kind="caution" title="Quotes pending">
  Three SMT assemblies contribute zero cost – the rollup is a lower bound.
</Callout>
```

7. Information layout & data presentation

- **Rule-based tables are the default for dense technical data.** Use a strong header rule and avoid unnecessary outer boxes.
- **Numeric columns are right-aligned** and use IBM Plex Mono with tabular figures where the platform supports them.
- **Use tables for structured multi-column data.** For only a few key/value pairs, plain aligned rows are usually clearer.
- **Do not wrap dense identifier/path cells by default.** Truncate or provide expansion where necessary.
- **Cards are for grouping, not decoration.** Use the Identity Guide's card/panel tokens; reserve accent outlines for meaningful active/current state.
- **Stat cards are for single key metrics.** They may carry a semantic tone, unit, and short sublabel; do not turn every value into a card.
- **One hero card at most** (§6).
- **Narrative text uses full sentences.** Explanatory prose should not become a wall of bullets when a short paragraph communicates the relationship better.

8. Data visualisation

Data visualisation is an interface, not decoration. Three palettes exist, each for one kind of surface.

8.1 Ops charts – the console plot colours

Tools, dashboards, BAC Ground, technical documents and reports use --chart-1 … --chart-6 in this fixed order, per theme:

Series	Light	Dark
1	#0B5CAD (blue)	#4C9AFF
2	#3F3F3F (text grey)	#E5E5E5
3	#B58F00 (ochre)	#F7D400
4	#1A7F37 (green)	#3FB950
5	#C9302C (red)	#FF5A4F
6	#8A8A8A (grey)	#6B6B6B

The set reuses the status hues, so the green/red rule is restated: **when a chart also carries pass/fail or limit meaning, do not use --chart-4 or --chart-5 as plain series** – use them only for the pass/fail marks and limits themselves, and extend the series with tints or stroke styles instead. Series 6 is for reference and background series (in dark mode it is below 3:1 on cards, so never the only carrier of an essential series).

8.2 Mission and marketing charts – the nebula triad

Mission collateral, marketing and presentation charts use Orbital Teal, Ion Violet and Plasma Rose, in that order. **At most three series**; beyond that, fall back to neutrals and tints of the three. Never use the triad in Ops charts.

8.3 Notebooks – the shared palette

Marimo notebooks cannot know whether their charts land on a light or a dark surface (marimo reports "light" under its "system" setting). They use one palette that reads on both: --chart-shared-1 … -5 = #2E7ED8, #7D7D7D, #9D7800, #0E903A, #E4463E, about 4:1 on white and on #1F1E1D – the console order (blue, grey, ochre, green, red) at mid lightness. Chart text, axes and ticks carry no colour of their own and take marimo's chart theme. The Project & Tooling Guide §4.5 has the implementation.

8.4 Encoding

Encode line meaning by stroke as well as colour: **solid** for observed data, **dashed** for targets and limits, **dotted** for estimates. Give each series a distinct marker shape when lines cross. On dark chart surfaces every series mark keeps at least **3:1 graphical contrast** (series 6 excepted, see §8.1). (This dual encoding is also an accessibility requirement; see §5.)

8.5 Chart construction rules

- Every chart has a clear title and explicit axis titles with units where applicable.
- Avoid decorative gridlines; use minimal ticks and reference lines only when they aid reading.
- Data that fits comfortably in a table of fewer than ~20 rows usually belongs in a table rather than a chart.
- A single value is not a chart. Present it as a stat/metric.
- Sequential and diverging schemes must preserve semantic meaning and contrast; do not force brand colours where they distort the data.

9. Feedback, progress, errors & confirmation

- **Liveness threshold:** any silent operation longer than ~1 second shows a spinner, progress bar, or equivalent activity indicator for its entire duration.
- **Determinate progress:** show the task name, progress, and one right-aligned status such as x/n, percentage, or elapsed time. Pick one, not all three. The bar fill is --accent-state in GUIs.
- **Indeterminate progress:** use restrained spinners/indicators or a loading shimmer. No novelty animation.

- **Completion:** replace or collapse transient progress output rather than leaving a wall of historical progress lines.
- **Errors:** remain visible, identify what failed, and give an actionable next step when one exists. Raw implementation traces stay hidden unless explicitly requested/debug mode is active.
- **Routine success:** confirm succinctly, then get out of the way.
- **Confirmation:** destructive or costly actions require an explicit confirmation or a clear before/after review. Low-stakes yes/no can use a simple confirmation; guided selection is appropriate for 4+ choices or whenever a wrong choice has real cost.
- **Dry runs/previews:** clearly label that no changes were made.

10. Terminal interface profile

10.1 Colour usage in Rich markup

Use hex colours directly in Rich markup. The values are generated from the BAC tokens into bac-common (bac_common.ui); use its constants rather than retyping hex:

Role	Rich markup	Hex
Success / pass / uploaded	[#3FB950]	#3FB950
Error / fail / blocked	[#E94538]	#E94538
Warning / alert	[#F7D400]	#F7D400
BAC Yellow accent (progress fill, inline bars)	[#F7D400]	#F7D400
Label / key name	[bold]	–
Muted / secondary	[dim]	–
Code / paths / identifiers	[cyan]	–

The terminal cannot tell a light theme from a dark one, so each role has one hex, tuned for dark terminals: on a dark terminal green is 6.6:1, red 4.3:1 and yellow 11.4:1; on a white terminal green (2.5:1) and especially yellow (1.5:1) are weak, which is accepted – the glyph and the word always carry the state too.

Keep active colours to a maximum of three per screen state. Reserve #F7D400 for the single most visually dominant yellow element currently on screen. Small semantic warnings or status markers may coexist when they carry genuine meaning; avoid multiple competing large yellow focal points.

Example – status line pattern:

```
console.print(f"  [{GREEN}]✓[/] Uploaded: [cyan]{item.public_url}[/]")
console.print(f"  [{RED}]×[/] Failed: [cyan]{item.blob_name}[/]")
console.print(f"  [{YELLOW}]![/] Warning: {message}")
```

10.2 Progress and feedback

- **The tool must always appear alive.** Any operation that takes more than ~1 second without producing output must show a spinner or progress bar for its entire duration. A silent terminal is indistinguishable from a crash; never leave the user guessing.
- Use rich.progress.Progress for any operation taking more than ~1 second.
- Progress bars show: task name, bar, and one right-aligned status (count x/n, percentage, or elapsed time – pick one, not all three). Style the bar fill with #F7D400.
- Spinners for indeterminate waits: dots, line, or bouncingBar. No novelty spinners.
- After completion, replace or collapse progress output. Do not leave a wall of progress lines in the scroll buffer.

10.3 Tables

- Use rich.table.Table for structured multi-column output.
- Default box style (Rich's HEAVY_HEAD) is the reference standard – bold header row underline, no outer border. This is what the reference implementation uses.
- Use box.SIMPLE for secondary or inline tables where less visual weight is appropriate.
- Column headers: bold (default in Rich tables).
- Tables wider than the terminal truncate rows intelligently; do not wrap cell content.
- Use tables when data has 2+ columns and 3+ rows. For fewer rows, plain key–value pairs are cleaner.
- Box-drawing characters naturally produce right angles in terminals; do not simulate rounded corners there.

```
from rich.table import Table

table = Table(title="Ingest plan")
table.add_column("Source")
table.add_column("Title")
table.add_column("Placement")
table.add_column("Confidence")
console.print(table)
```

10.4 Inline bar charts

Horizontal bar charts built from block characters are acceptable for quick relative comparisons. Do not use a third-party terminal charting library for this; a hand-rolled block chart using Rich markup is more controllable and has no extra dependency.

Rules: one chart per output block; decorative aid only, not a dashboard.

```
Uploaded resources
Datasheets  [#F7D400]████████████████[/][dim]░░░░[/]  42
Schematics  [#F7D400]██████[/][dim]░░░░░░░░░░[/]  14
Firmware    [#F7D400]███[/][dim]░░░░░░░░░░░░░░[/]   7
```

Filled blocks: #F7D400. Empty blocks: dim white.

10.5 Panels and headers

- Use rich.panel.Panel for two things only: the opening run summary and the final result block.
- Section dividers: rich.rule.Rule with a short label. No decorative banners.
- Opening panel should identify the tool, version, and what is about to happen:

```
from rich.panel import Panel
from rich import box as rich_box

console.print(
    Panel(
        "[bold]bac-toolname[/bold]  [dim]v0.1.0[/dim]\nOne-sentence description of what this run will do.",
        box=rich_box.SIMPLE,
        border_style="dim",
    )
)
```

The canonical branded header line (for use inside the panel or as a plain text fallback) is:

```
Build a CubeSat – toolname v0.1.0
```

10.6 Structural layout of a CLI run

A typical tool run follows this sequence:

```
[Panel] Tool name  version · brief description

[Progress or step log]
  Found 3 input document(s).
  [#3FB950]✓[/] Uploaded: https://...
  [#3FB950]✓[/] Uploaded: https://...
  [#E94538]×[/] Failed: document.pdf – reason

[Table or Rule] Result summary
  Processed : 3
  Uploaded  : 2
  Errors    : 1
```

- **Opening:** one panel, shown once.
- **Step log:** one console.print() per logical step. Successes may be dim after confirmation; errors stay visible.
- **Result summary:** always shown, even on partial failure. Numbers; no prose narrative.
- **Dry run:** append [dim](dry run – no changes made)[/dim] to the result summary.

11. Web & notebook interface profile

These rules apply to public web interfaces, internal web tools, the ground console, and reactive notebook UIs. Framework-specific implementation belongs in the Project & Tooling Guide.

- Put user-adjustable inputs in a **single, discoverable control region** instead of scattering controls through unrelated content.
- Every input has a visible label. Units belong next to values (CHF, %, mm, etc.).
- For 2–3 short mutually exclusive choices, radio/segmented controls are appropriate; for 4+ choices, prefer a select/dropdown.
- Use switches for boolean states and checkboxes for inline inclusion/exclusion where that distinction is clear.
- Use file-upload controls only when external input is actually part of the workflow.
- Use sortable/filterable tables for inspection and dataframes/explorers for deeper analysis.
- Put verbose intermediate output in expandable/accordion regions so users can inspect it without scrolling past it every time.
- Important results or warnings use callouts; single key metrics use stat treatments; the one most important item may use the hero card.
- Icons are Lucide, embedded inline (Identity Guide §8); icon buttons use 18px icons.

12. Physical indicators (LEDs)

Physical status LEDs are a UI surface like any other – the semantics below extend the same state system used everywhere else in this document, so a board's indicator lights, the console and a tool's terminal output read the same way.

12.1 Single-colour LEDs

Color	Meaning	Behavior
White	Power / board present	On whenever the board has power – the "it's alive" indicator (mirrors PGOOD).
Green	Nominal / pass	Steady on = healthy operating state. Same meaning as the nominal state (--bac-green).
Red	Fault / alarm	Steady on = latched fault. Reserved strictly for real problems – never dual-purpose. Same meaning as the alarm state (--bac-red).
Yellow	Caution / degraded	E.g. approaching current limit, low battery, suboptimal comms mode. Same meaning as the caution state (--bac-yellow); there is still no amber LED role – yellow covers it.

Color	Meaning	Behavior
Blue	Activity / communication	Blinks on data transfer or bus traffic – the information/activity meaning of the info state. Deliberately kept separate from the health colors (white/green/red/yellow) so it can flicker freely without being read as a state change.

Discrete LEDs are fixed-wavelength parts, not tunable RGB – exact hex matching to the digital tokens isn't the goal. Pick the closest standard wavelength (roughly 565–575 nm green, 588–590 nm yellow/amber, 620–630 nm red) rather than sourcing an unusual bin to chase a hex value.

12.2 Tri-colour (R/Y/G) indicator

Use a single red/yellow/green package as the one-glance **health** indicator for a board or module – one place to look instead of three separate LEDs:

- **Green** – nominal.
- **Yellow** – caution / degraded.
- **Red** – fault.

Same traffic-light mapping as the singles, just consolidated. A board may carry both: white for power, tri-color for health – they answer different questions ("is it on" vs. "is it okay").

12.3 Behaviour & disambiguation conventions

- **Solid = steady state. Blinking = transient or in-progress** (e.g. blinking green during boot/self-test, solid green once nominal).
- **Faults latch.** A red fault stays lit until cleared or the board is power-cycled – never let a transient fault self-clear the LED, or a real problem goes unnoticed.
- **Document "off."** Unpowered, disabled, and no-traffic can all look identical unlit; the README/datasheet for each board must state what an unlit LED means for each indicator.
- **Colour is never the only state channel.** Physical hardware may disambiguate state through **behavior, position, or typography**. Separate LEDs should be identified by silkscreen or another persistent label such as PWR, OK, WARN, FAULT, ACT, or equivalent; their physical position plus the label provides a non-colour cue. Distinct blink patterns may be used as an additional channel.
- **A shared multi-colour indicator needs a dynamic second channel.** Silkscreen such as STATUS identifies the indicator's purpose but cannot reveal which colour is currently lit. If a single R/Y/G LED uses colour to distinguish nominal, caution, and fault, use distinct temporal patterns where practical or expose the same state through a labelled

display, terminal, log, or other interface. Do not claim a shared multi-colour LED is independently accessible when all states use identical steady behaviour.

- **Label the meaning, not merely the colour.** Prefer FAULT or WARN over RED or YEL where PCB space permits. The label should communicate what the indicator means even if its hue cannot be distinguished.

13. Reference applications

- **BAC Ground** – the reference implementation: the five-state system, pill controls, flat cards, the wide container, ops plot colours, one hero card on the overview, grey actions in dark mode.
- **Storefront** – combines the Identity Guide's Mission presentation with standard accessible controls, links, forms, and light/dark behaviour.
- **Pricing tool and engineering notebooks** – Ops interface: one control region, stat cards for key figures, rule-based data tables, the shared notebook chart palette, and no decorative Mission texture.
- **Product sheet** – primarily a communication surface governed by the Identity Guide; any linked/interactive digital version still follows this guide for controls, focus, contrast, and semantics.
- **CLI tools** – terminal profile in §10, implemented according to the Project & Tooling Guide.
- **Hardware boards** – physical indicator profile in §12; firmware implementation must preserve those semantics.

14. Quick reference – the non-negotiables

1. Minimal surface, maximum clarity. Fail loudly, succeed quietly.
2. Show liveness for silent operations longer than ~1 second.
3. Every state is colour + shape + word: nominal (green, filled circle), caution (yellow, triangle), alarm (red, square), info (blue, diamond), no data/planned (grey, dashed hollow circle).
4. Status colours mean state and nothing else; functional warnings use BAC Yellow, never an amber or a nebula colour.
5. One visually dominant yellow action/surface per state in light mode; small legitimate semantic indicators may coexist.
6. One standard dark mode: grey primary buttons, neutral state marks, yellow only as caution. A user accent setting is allowed when state never depends on the accent colour.
7. Pill controls with uppercase mono labels; 10px cards; no shadows, no lifts; hover is a surface step.
8. Light and dark modes are first-class for GUI/web; terminals inherit the user's theme.

9. Motion is restrained, functional, and reduced when prefers-reduced-motion is set.
10. Ops charts use the console plot colours, Mission charts the nebula triad (max three series), notebooks the shared palette; always dual encoding with stroke or marker.
11. Tables are rule-based and information-dense, numbers in mono tabular figures.
12. At most one hero card per view; its red rule is decorative.
13. Destructive/costly actions require explicit confirmation or review.
14. Physical LEDs share digital state semantics. Separate LEDs may use silkscreen/position as the non-colour channel; a shared multi-colour status LED needs an additional dynamic channel.
15. Identity tokens and Ops/Mission choice come from the BAC Identity Guide; implementation belongs to the BAC Project & Tooling Guide.

15. Version history

Version	Date	Change
1.0	2026-10-27	Rebased on the BAC Ground console, with Mänu as design owner and Gui as co-author (decided 2026-10-24 to 2026-10-27). §1: status colours mean state and nothing else. §2: the five-state system with fill, ink, shape and word, and the F Prime severity mapping; accent and action colours per theme; one standard dark mode with grey primary buttons and neutral state marks (--accent-state), an optional user accent setting, yellow in dark mode only as caution; blue is info, links and focus (#4277FD retired). §3: wells lighter than the page in dark mode; the two container widths. §4: no lifts or shadows, hover as a surface step; the focus ring at #0B5CAD/#4C9AFF with recomputed ratios and no halo. §5: tabular figures. §6: pill buttons with uppercase mono labels, round 18px Lucide icon buttons, pill tabs and segmented controls, status chips as colour + shape + word, and the new hero card with its decorative red rule. §8: three palettes – the console plot colours for ops charts with the green/red rule restated, the nebula triad for Mission charts (max three series), the shared mid-tone palette for notebooks. §10.1: Rich colours #3FB950 / #E94538 / #F7D400, generated into bac-common. §12: LED meanings tied to the state names. §13, §14 rewritten. The authority note points to the BAC Trademark Policy for the logo's use by others.
0.2	2026-10-24	§3: light is the default when the system reports no preference, since CSS cannot tell that case from light. §4.2 and §5: the focus indicator is a 3 px solid --focus-ring outline with a 2 px offset; --shadow-focus is a halo, not the indicator. §8.1: the derived series for dark chart surfaces as the reference gallery uses them.
0.1	2026-08-10	Created by splitting interface behaviour out of the former BAC Design System Specification v0.6 and BAC Internal Tools Design Language Guide v0.10. Consolidated motion, interaction states, components, accessibility, data-viz encoding, terminal presentation, feedback/progress, web/notebook UI patterns, and

Version	Date	Change
		physical indicators. Clarified one-dominant-yellow usage, added dark-chart contrast guidance, and explicitly allowed silkscreen typography/position to disambiguate separate LEDs while requiring a dynamic second channel for shared multi-colour indicators.